



Sapienza University of Rome



Fit Pose: Real-time Fitness Tracking Using MediaPipe

Carmine Fabbri – 2133421

Giuseppe Giuliano – 1973880



Pose Estimation and Virtual Gym Assistant using MediaPipe and ML

2023 International Conference on Network, Multimedia and Information Technology (NMITCON)

Problem statement

Physical activity is essential in today's society since it helps build stronger bones and muscles, which enhances our capacity to carry out daily tasks.

Lack of access to professional gym trainers leads to incorrect exercise forms and injuries, especially during home workouts.

Objective

This research paper, demonstrates a virtual fitness trainer, that monitor the user's every step to provide a useful feedback and insights as they work out.

The system determines the user's performance by computing different angles and landmarks.





Technology and Main Features



MediaPipe

Mediapipe's pose detection API uses **BlazePose**, which is a lightweight neural network with high accuracy and low computing complexity.

Landmark detection

The pose detection API for landmark detection offered by BlazePose, is able to detect up to **33 landmarks**.

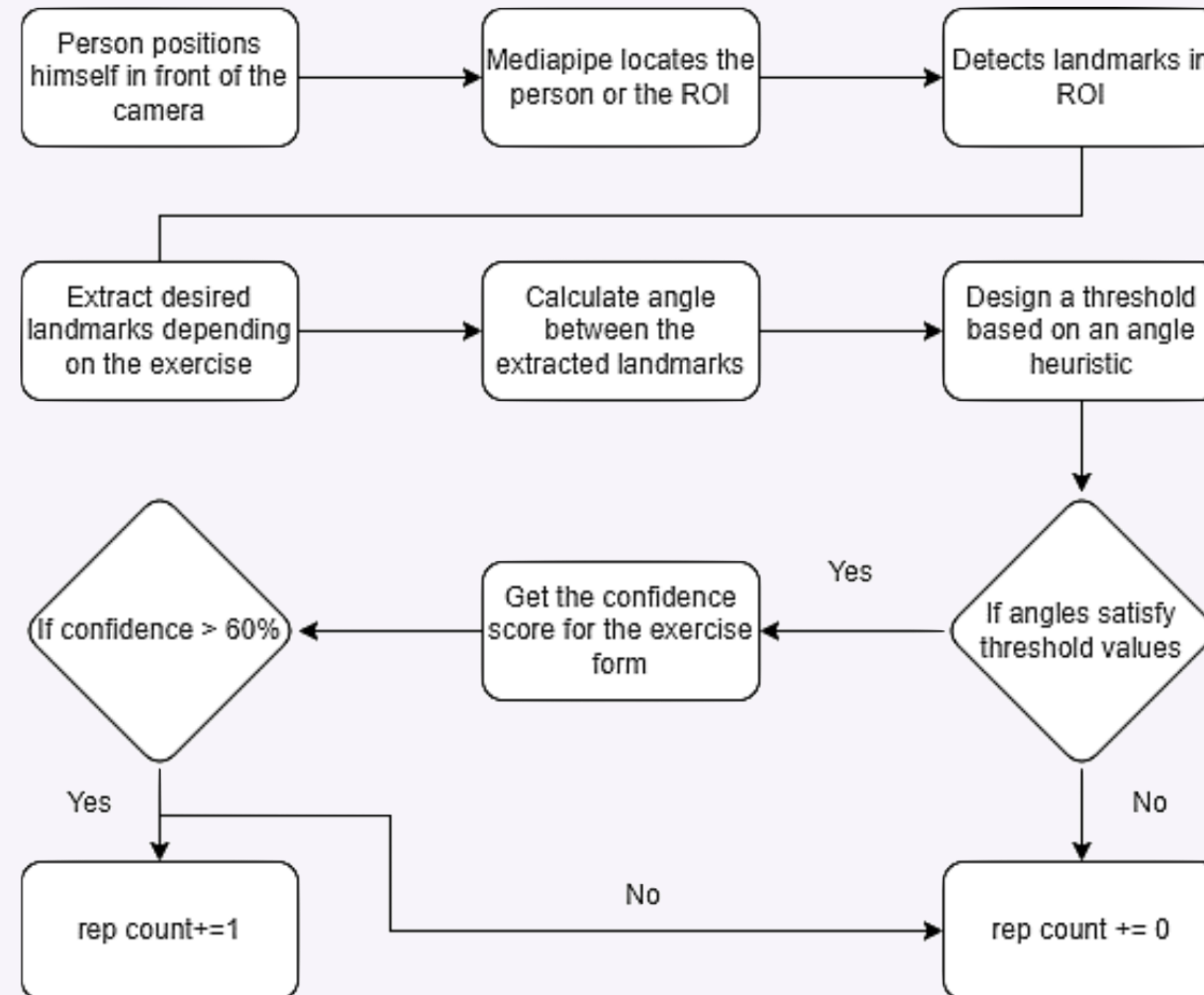
Machine Learning

Researchers have used Machine Learning model to validate only **one kind** of exercise, which is biceps curl

Reps counting stage

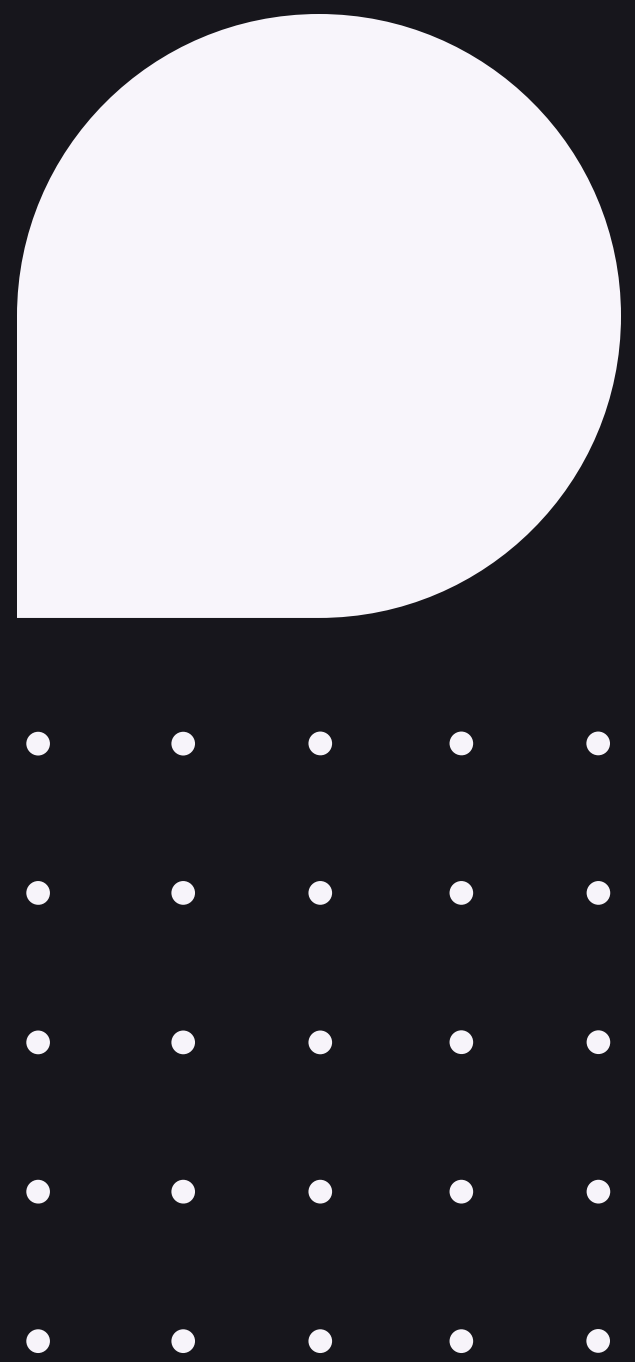
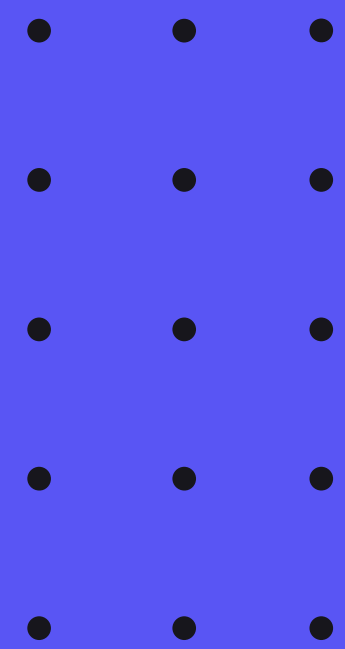
Through the use of different heuristic thresholds, the researchers were able to validate the different exercises.

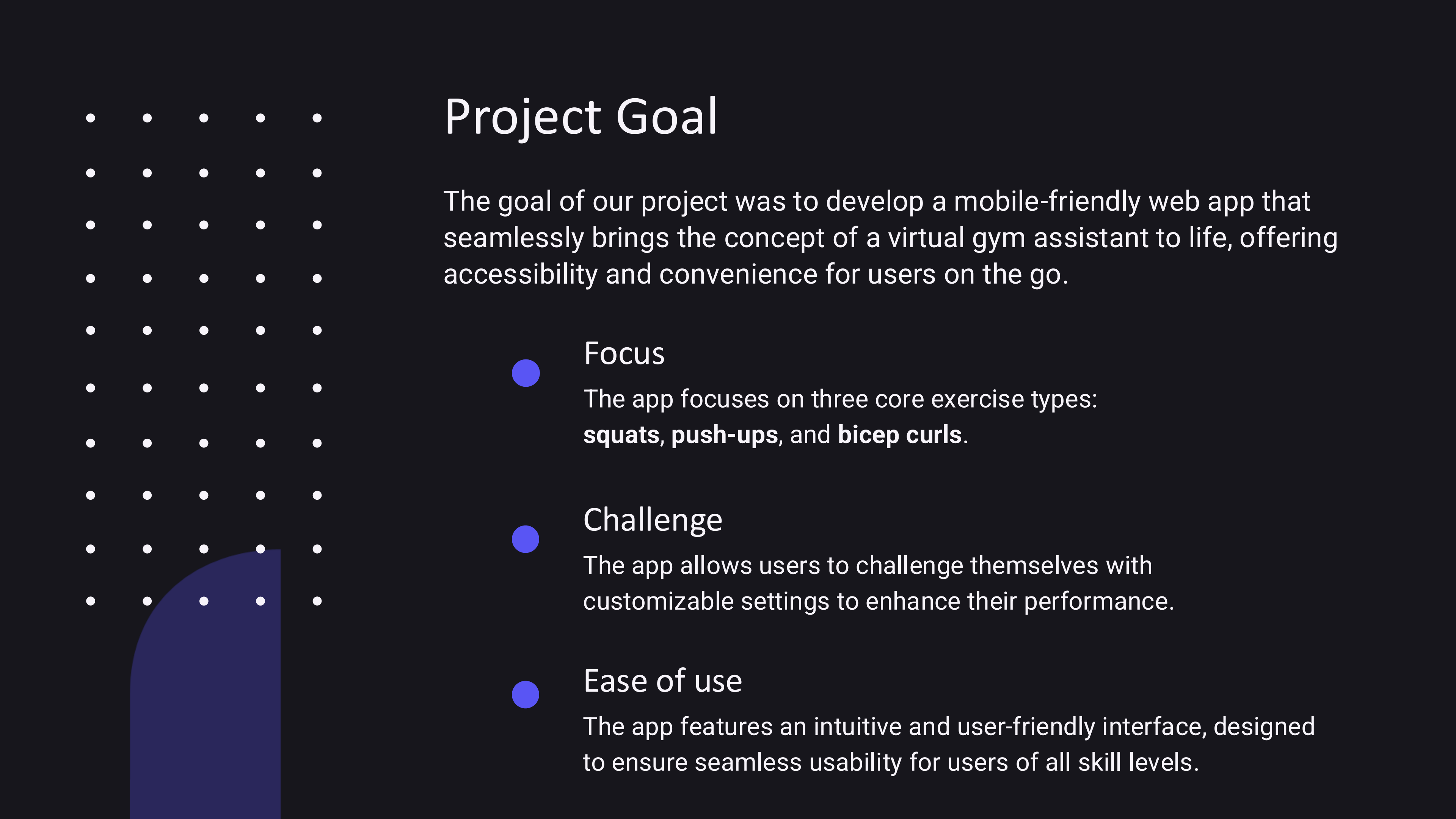
Proposed Approach



Process Pipeline from the paper

Our solution:
Fit Pose





Project Goal

The goal of our project was to develop a mobile-friendly web app that seamlessly brings the concept of a virtual gym assistant to life, offering accessibility and convenience for users on the go.

Focus

The app focuses on three core exercise types: **squats**, **push-ups**, and **bicep curls**.

Challenge

The app allows users to challenge themselves with customizable settings to enhance their performance.

Ease of use

The app features an intuitive and user-friendly interface, designed to ensure seamless usability for users of all skill levels.

Technology Used

MediaPipe

MediaPipe's Pose Detection API leverages **BlazePose**, a lightweight and highly accurate neural network optimized for low computational complexity.

Tensorflow

It provides tools for building, training, and deploying models for various tasks, including computer vision. Its flexibility for multiple platforms make it ideal for real-time AI applications.

React.js

React enables the development of fast, dynamic, and responsive web applications. Its component-based architecture simplifies code management and reuse, while the virtual DOM ensures efficient rendering for smooth user experiences.

Reps counting stage

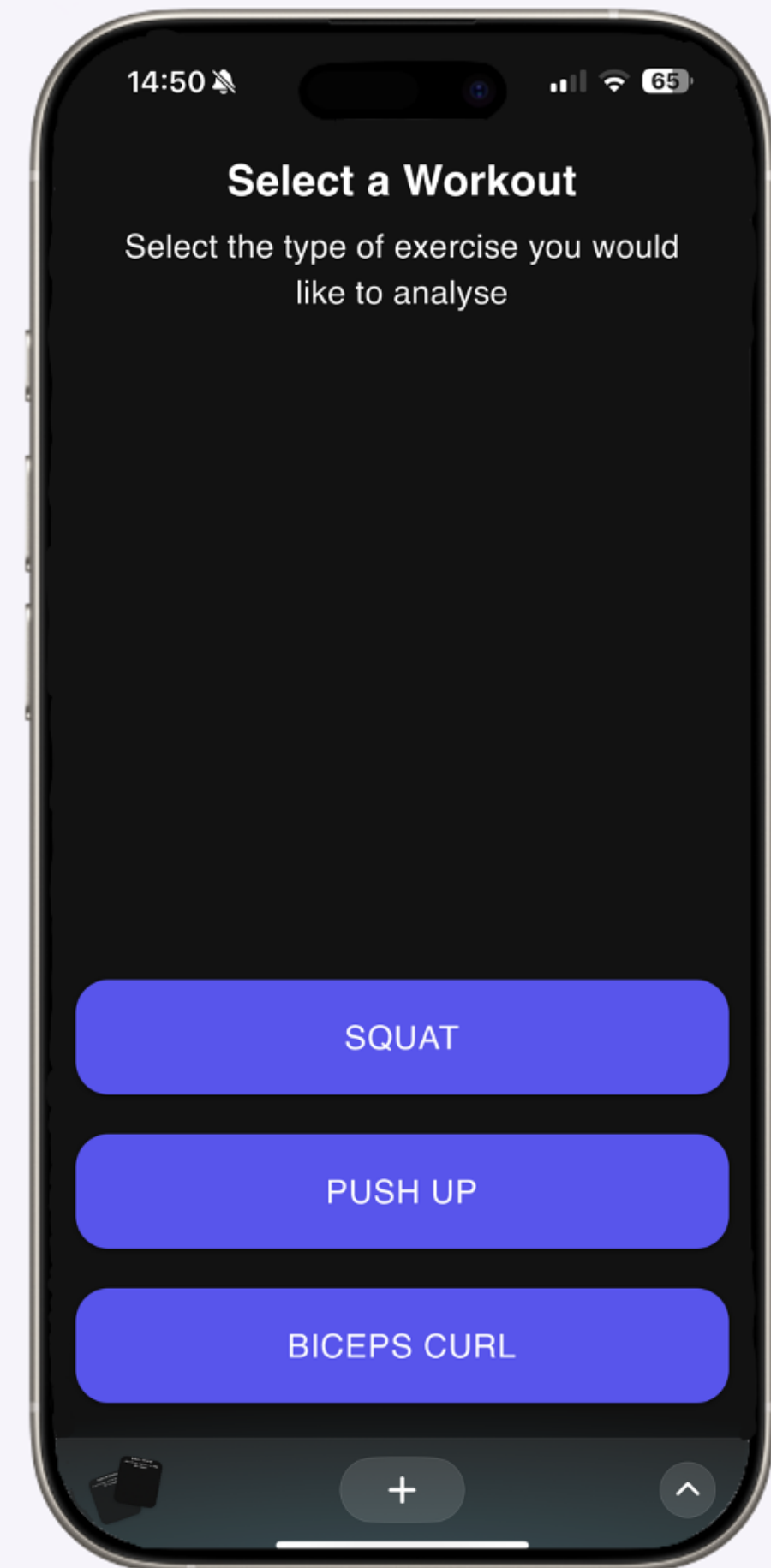
We evaluate each exercise with its own specific heuristic thresholds.





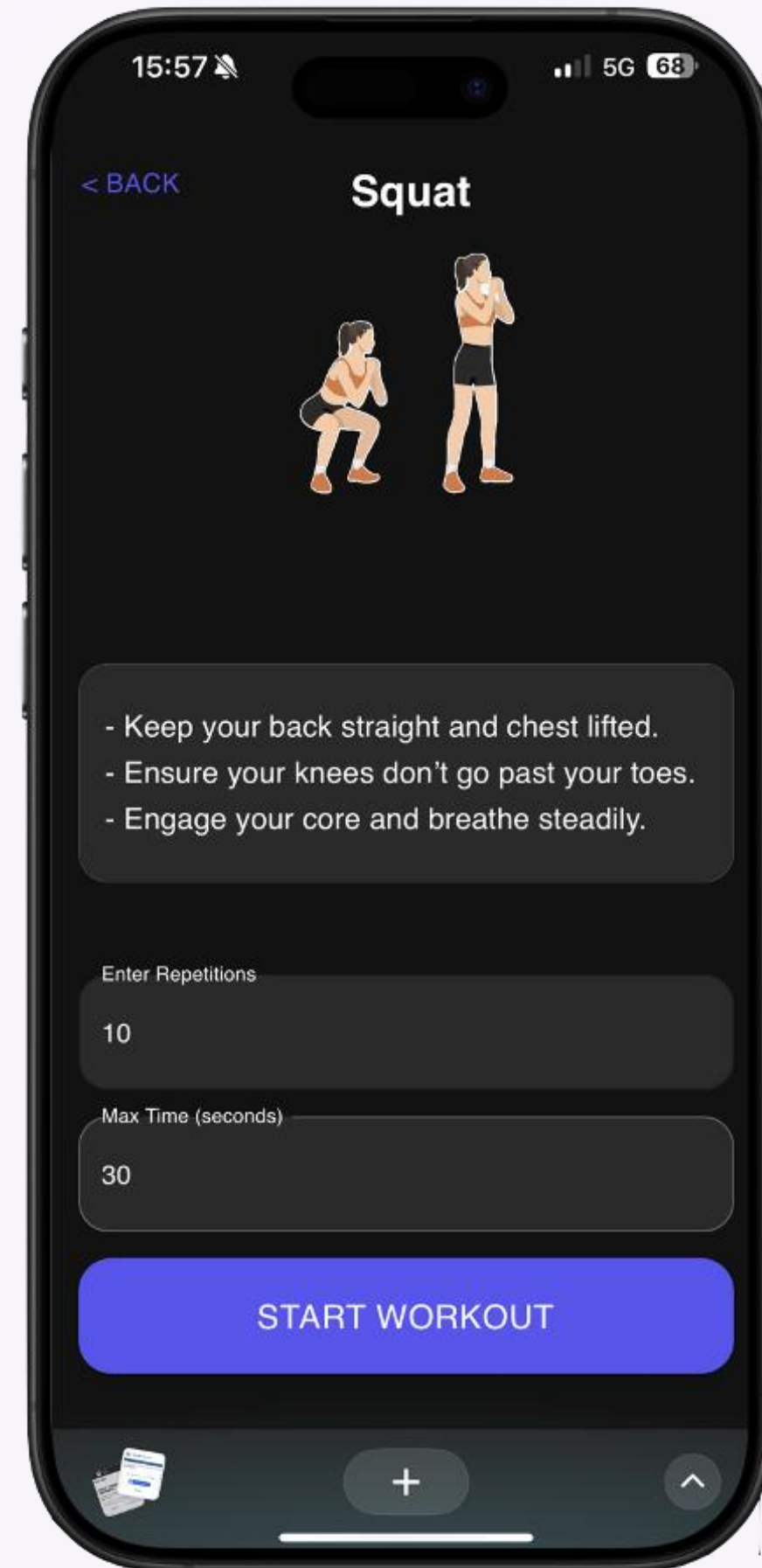
Workout selection

The user can easily select between **Squat**, **Push-Up** and **Biceps Curls** exercises to begin tracking reps and improving performance.



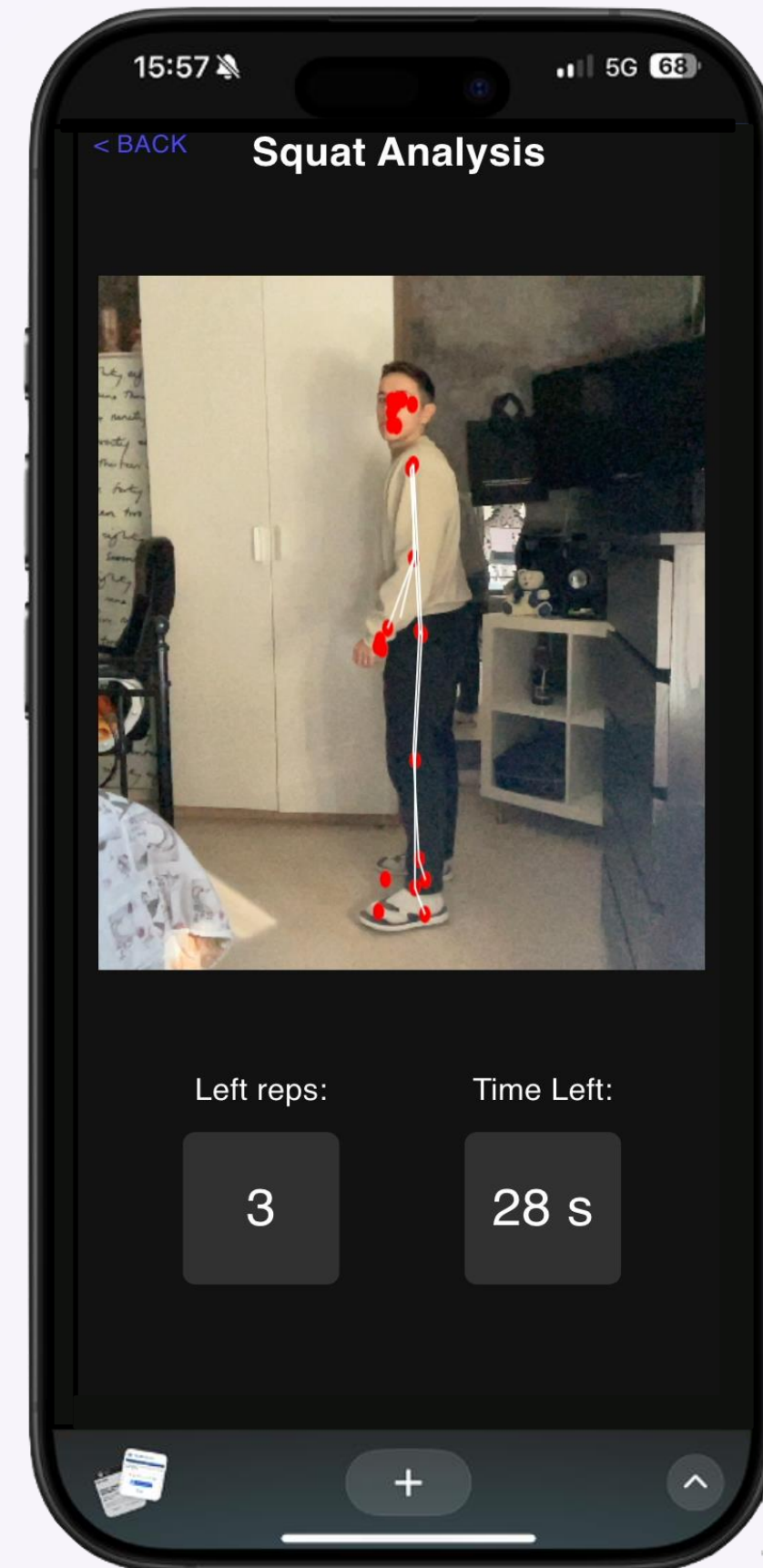
Challenge yourself

The user can challenge himself by setting the time and rep goal, which represent how many reps he want to complete within a specific time frame



Landmark detection

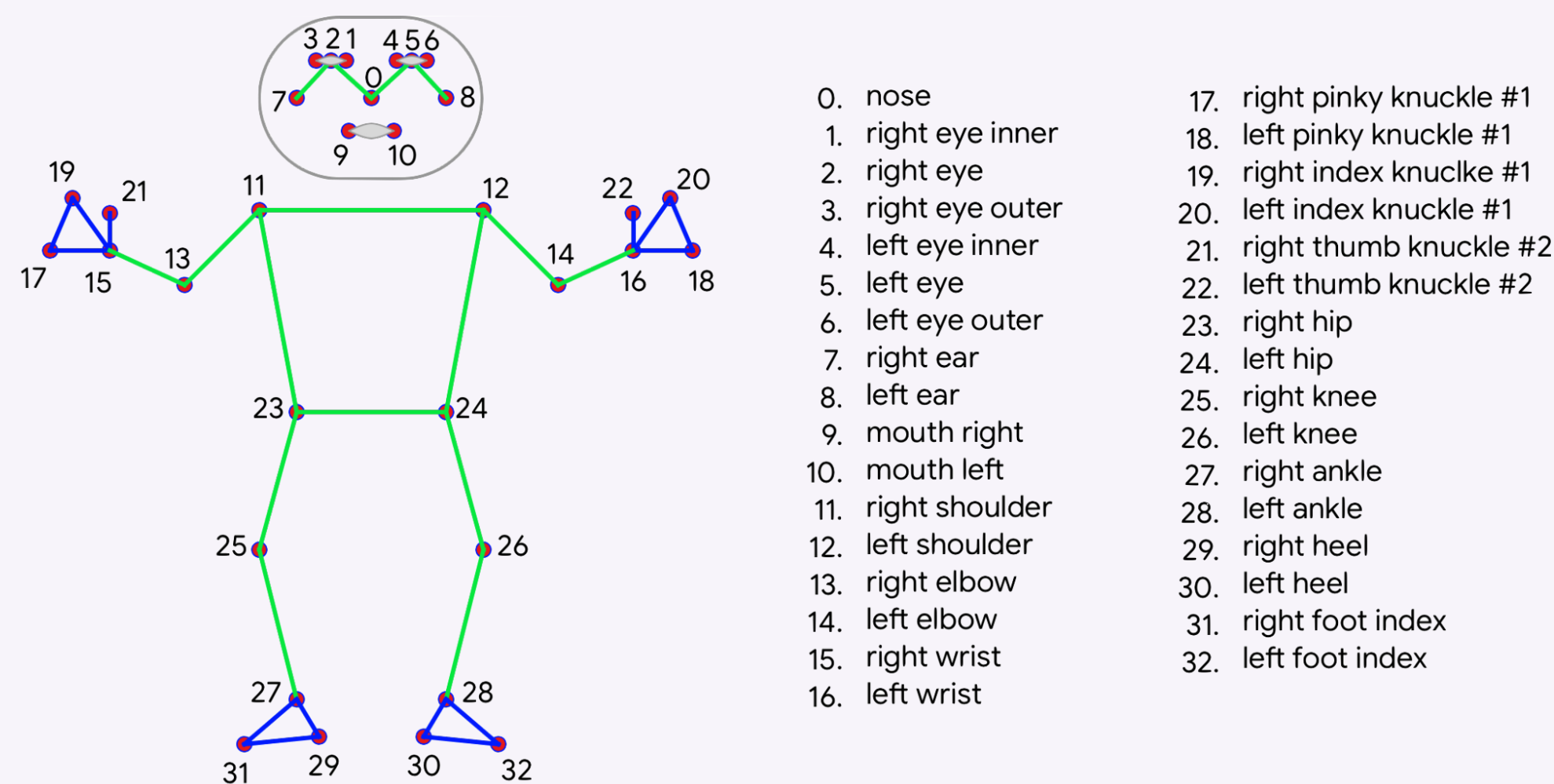
The user can position themselves in front of the webcam, allowing the system to start tracking their movements using pose estimation and body landmarks to verify exercise correctness and count repetitions in real time.



BlazePose Landmark map

The current standard for human body pose is the **COCO** topology, which consists of 17 landmarks across the torso, arms, legs, and face.

With *BlazePose*, we can use a new topology of **33 human body keypoints**, which is a superset of COCO, which allow to determine body semantics from pose prediction alone that is consistent with face and hand models.



Landmarks of MediaPipe BlazePose Model

Squat Detection

The squat detection algorithm analyzes pose landmarks to track the vertical distance between key body points, like head, ankle and knees, in real time. By applying thresholds to normalized and smoothed distances, it identifies when a squat starts and ends. An AI model further validates the movement's correctness, ensuring accurate and reliable squat counting for fitness applications.

1 Input

The algorithm takes 33 pose landmarks as input, each containing (x, y, z) coordinates and a visibility score.

2 Detection

The algorithm detects squats by calculating the normalized vertical distance between the head and ankle and by also considering the angle of the knee during the execution.

3 AI Validation

A custom designed *TensorFlow* model analyzes preprocessed landmarks to validate squat correctness, ensuring accurate counting and filtering out improper movements.

Push-up Detection

We calculate the angle between key points (**head**, **ankle**, **wrist**) and measures the distance between the head and wrists to evaluate motion.

A deep learning model predicts the correctness of each repetition, ensuring accurate counting and feedback. The model processes flattened landmarks as input and identifies whether the push-up meets form standards. This combination of pose estimation and AI provides real-time feedback for improving exercise performance.

1

Detection

The system uses MediaPipe to detect key body landmarks in real-time, including the head, wrists, ankle, and shoulders. These landmarks are analyzed to calculate angles and distances, enabling precise tracking of push-up movements and posture

2

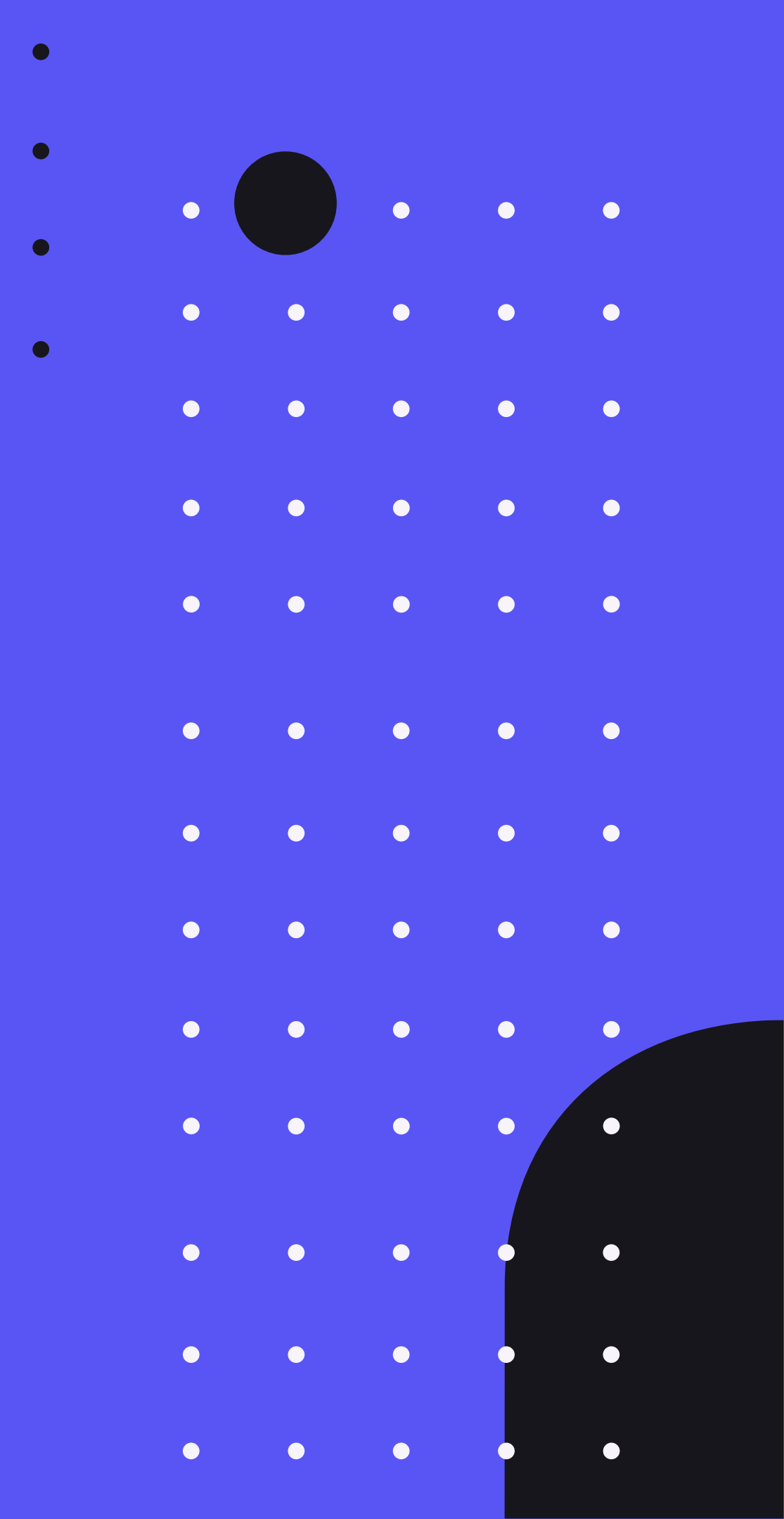
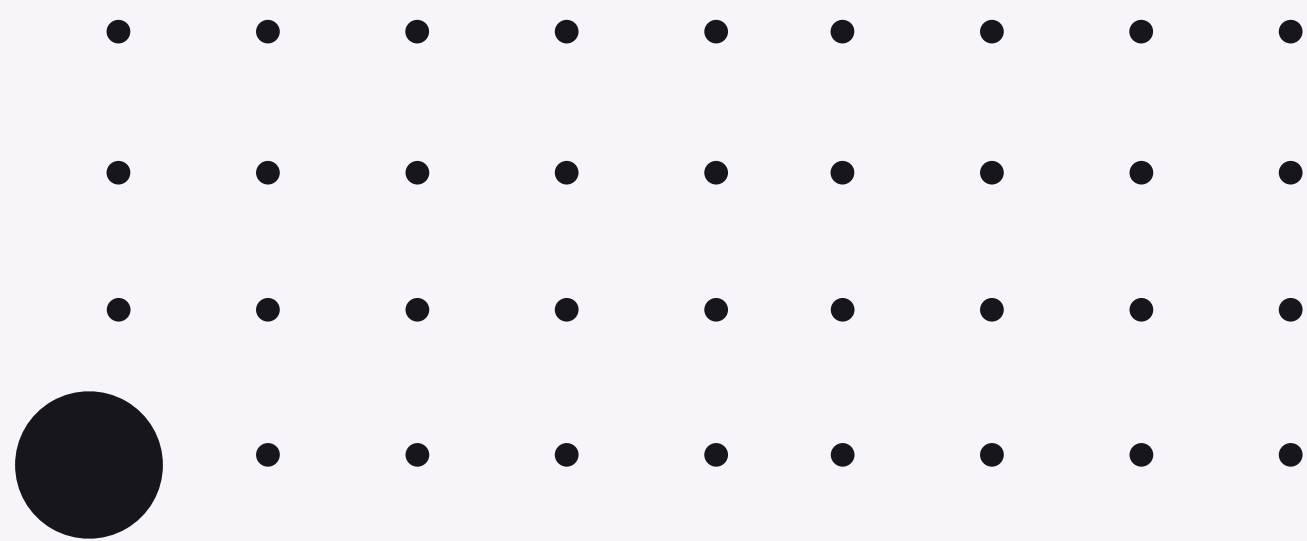
AI Validation

The model validates the data by processing flattened pose landmarks as input, predicting correctness based on learned patterns of proper push-up form, and ensuring the prediction confidence exceeds a defined threshold for accuracy.

Model Architecture

The model for **squat exercise** uses an **LSTM** architecture to process sequential *keypoints* data from 191 frames per video, with 132 features per frame. It includes dense layers for feature extraction and classification, using *softmax* activation for “**Valid**” or “**Invalid**” exercise prediction. Trained with binary cross-entropy and the Adam optimizer, the model achieves high accuracy and is optimized for real-time validation.

```
1 # base LSTM model used for our exercise prediction
2 model = Sequential()
3 model.add(LSTM(64, return_sequences=False, activation='relu', input_shape=(191, 132)))
4 model.add(Dense(64, activation='relu'))
5 model.add(Dense(32, activation='relu'))
6 model.add(Dense(Squat_result.shape[0], activation='softmax'))
7
8 model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
```



Thank you!

[LINK TO GITHUB](#)

[LINK TO THE PAPER](#)

